

AUDITORÍA TÉCNICA — LinkServi

Diagnóstico estructural del sistema actual basado en el código real. Fecha: 26 abril 2026 · Repo: `linkservi/` (pnpm monorepo).

1. ARQUITECTURA GENERAL

Tipo de app: SPA pura (Single Page Application) con backend REST separado. **No hay SSR.**

Estructura del monorepo (pnpm workspaces):

- `artifacts/servilink/` — Frontend React 19 + Vite 7 (la app que ve el usuario).
- `artifacts/api-server/` — Backend Express 5 + Socket.io.
- `lib/db/` — Esquema Drizzle ORM compartido.
- `lib/api-spec/` — Especificación OpenAPI + Orval.
- `lib/api-client-react/` — Hooks React autogenerados desde el OpenAPI.
- `lib/api-zod/` — Schemas Zod compartidos generados.
- `artifacts/mockup-sandbox/` — Sandbox de componentes (no productivo).

Flujo usuario → DB:

- Browser carga el bundle Vite (SPA, ruteo client-side con Wouter).
- Componentes hacen `fetch('/api/...')` con header `Authorization: Bearer <jwt>` y cookie `sl_token`.
- Reverse proxy de Replit enruta `/api/*` al `api-server`.
- Express aplica middlewares: `pino-http` → `helmet` → `cors` → `express.json` → `cookieParser` → `rate-limit` → `authenticate` → ruta de dominio.
- Ruta usa Drizzle ORM (queries parametrizadas) contra PostgreSQL 16.
- Eventos en tiempo real (chat, disputas) viajan por Socket.io.

Comunicación entre capas: todo HTTP/JSON + WebSocket. Auto-generación con Orval mantiene el contrato API alineado entre back y front.

2. TECNOLOGÍAS USADAS

Frontend (`@workspace/servilink`):

- React `19.1.0`, Vite `^7.3.2`, TypeScript `~5.9.2`.
- Ruteo:** Wouter `^3.3.5` (no React Router).
- Estado servidor:** TanStack Query `5.90.21` (`staleTime: 60s`, `gcTime: 10min`).
- Estado UI:** React Context (`AuthContext`, `SidebarContext`, `ThemeContext`).

- **UI:** Tailwind CSS `4.1.14` + shadcn/ui (componentes en `src/components/ui/`) + Framer Motion.
- **Mapas:** Mapbox GL `^3.22.0` y React Leaflet (mezcla).
- **Auth social:** Firebase (Google login).
- **Realtime:** `socket.io-client`.
- **PWA:** `vite-plugin-pwa` con Workbox (`src/sw.ts`).

Backend (`@workspace/api-server`):

- Express `^5.0` (sí, versión 5, no la 4).
- Socket.io `^4.8.3`.
- Logging: `pino` `^9` + `pino-http`.
- Auth: `jsonwebtoken` + `bcryptjs` (10 rounds).
- Build: `esbuild` (bundle ESM en `dist/index.mjs`).

Base de datos:

- PostgreSQL 16 (módulo Replit).
- Drizzle ORM `0.45.2` (no usan migraciones manuales: `db push`).

Adicionales:

- Almacenamiento: Google Cloud Storage vía signed URLs (`routes/storage.ts`).
- Email transaccional: Resend (integración Replit).
- Notificaciones push: web push subscriptions.
- Pagos: API directa de Banco de Venezuela (BDV) C2P / Pago Móvil.

3. INFRAESTRUCTURA

Despliegue: Replit Deployments. Configurado en `.replit` con módulos `nodejs-24` y `postgresql-16`.

Servido de rutas:

- Frontend: SPA, todas las URLs caen en `index.html` y Wouter resuelve client-side.
- Backend: API REST bajo `/api/*` y endpoints SEO (`/sitemap.xml` , `/robots.txt`).

Limitaciones reales detectadas:

- **No hay redirects 301 en servidor.** Cualquier cambio de URL pública requiere lógica client-side, lo que es invisible para Google y rompe SEO de páginas migradas.
- **No hay CDN dedicado** (ni Cloudflare ni similar delante). El service worker mitiga, pero el primer request siempre golpea Replit.
- **Bundle estático servido por el mismo proceso** que la API en producción → competencia por CPU bajo carga.
- **Imágenes usuario subidas a GCS sin transformación on-the-fly** (no hay redimensionado al vuelo).

Riesgos de escalabilidad:

- Ranking de profesionales (`workers.ts` → `computeSmartScore`) ejecuta varias subqueries paralelas por request. Funciona hoy, se vuelve costoso a partir de ~50K profesionales activos.
- Listado de productos ordena en memoria del proceso (carga todos los activos y luego ordena en JS). Rompe a partir de ~10K productos.
- Socket.io en un solo nodo: sin Redis adapter, no escala horizontalmente.
- Sin colas de trabajos (`BullMQ` /similar): emails y procesamiento async corren in-process.

4. SISTEMA DE PAGOS

Núcleo bien diseñado, mezcla automatización C2P + verificación manual de comprobantes.

Archivos clave:

- `artifacts/api-server/src/routes/bdv-payments.ts` — C2P + webhooks BDV.
- `artifacts/api-server/src/routes/bdv-conciliacion.ts` — auto-verificación contra banco.
- `artifacts/api-server/src/routes/withdrawals.ts` — retiros de profesionales.
- `artifacts/api-server/src/routes/servicios/bookings.ts` — escrow + liberación.
- `artifacts/servilink/src/components/payments/C2PModal.tsx` — UI cliente.
- `lib/db/src/schema/bdv_payments.ts` — `bdv_c2p_transactions` , `bdv_payment_notifications` .

Flujo C2P (Pago Móvil):

1. Cliente ingresa cédula → backend pega a BDV `/paymentkey/v2` con header `X-API-Key: BDV_API_KEY` .
2. BDV envía SMS con OTP al teléfono registrado.
3. Cliente ingresa OTP → backend ejecuta el débito en BDV.
4. Si BDV responde código `1000` → `applyDomainEffect` actualiza booking/order/plan a pagado **automáticamente, sin admin**.
5. `referencia` y `endToEndId` quedan guardados en `bdv_c2p_transactions` .

Comprobante manual (alternativa):

- Cliente sube screenshot → estado `payment_pending` .
- Admin revisa y confirma/rechaza desde el panel.
- También existe verificación auto contra notificaciones BDV con tolerancia 10% por drift de tasa.

Liberación del dinero (escrow real):

- Pago capturado → fondos retenidos por LinkServi, NO acreditados al profesional.
- Booking pasa a `completed` (cliente confirma o auto-cron tras 25 min sin disputa) → fondos suman a `worker.earnings` .
- Profesional pide retiro vía `withdrawals.ts` → estado `pending` → `approved` → `paid` . Deducción atómica al momento del request (no double-spending).

Manejo de errores e idempotencia (esto está bien hecho):

- **Postgres advisory locks** (`pg_advisory_xact_lock`) basados en hash `referenceType+referenceId` durante el call al banco — previene condiciones de carrera.
- Webhooks BDV deduplicados por `referenciaBancoOrdenante` UNIQUE.
- Si el débito en banco fue exitoso pero la actualización de DB falló → llamada automática a `/annul` para reembolsar al cliente.
- Sin reintentos automáticos C2P; el usuario reintenta manualmente.

Seguridad del sistema de pagos (también bien):

- Webhooks BDV protegidos por `BDV_WEBHOOK_API_KEY` en header `X-API-Key`.
- **El servidor recalcula el monto esperado en USD desde su propia DB** y lo convierte a VES con tasa BCV vigente. **Ignora el monto del cliente** salvo sanity check (2% tolerancia). Esto blindo contra tampering del frontend.
- Replay protection vía referencias bancarias únicas en DB.

Riesgo: el `applyDomainEffect` ocurre dentro del mismo request HTTP del banco. Si BDV demora >30s y el cliente cierra el navegador, el flujo de fallback depende del webhook. Si el webhook nunca llega, queda pago en banco sin acreditar (mitigado por conciliación, pero no es transparente al usuario).

5. SISTEMA DE ROLES

Roles existentes:

- `client` — usuario que contrata servicios o compra.
- `worker` — profesional (UI dice "Profesional", identificador interno sigue siendo `worker`).
- `seller` — vendedor de tienda.
- `cohost` — copropietario/anfitrión de tienda.
- `admin` — sub-roles `super_admin`, `soporte`, `finanzas` (vía columna `adminRole`).
- `manager` — recién agregado vía tablas `business_managers` / `manager_invitations` (Fase 1 en progreso).

Almacenamiento (problema real):

- **Legacy:** `users.role` (text) + `users.secondaryRole` (text).
- **Moderno:** `users.roles` (text array) — es la fuente de verdad declarada.
- **Coexisten las tres columnas.** Cualquier check inconsistente entre código nuevo y viejo puede dar lecturas distintas.

Validación y enforcement:

- `authenticate` — middleware que llena `req.user` desde JWT (cookie `sl_token` o header Bearer).
- `requireRole(...roles)` — chequeo genérico (`auth.ts:123`).
- `requireAdminRole(...subRoles)` — admin con permisos finos.

- `requireManagerOf(storeId, permission)` — RBAC para gestores con permisos JSON.
- `userHasRole(user, role)` — helper que mira `roles[]`, `role` y `secondaryRole` (compatibilidad).

Riesgos detectados:

- **IDOR en** `GET /api/bookings/:bookingId` (`servicios/bookings.ts:341`): no valida ownership. Cualquier autenticado que adivine un ID ve detalles de bookings ajenos. **Esto debe arreglarse.**
- Tres columnas de rol coexistiendo es bug-magnet: si un trozo de código actualiza `role` y otro lee `roles[]`, el usuario "pierde" un rol sin saberlo.
- `requireManagerOf` lee permisos desde JSON string (no `jsonb`) → cada check parsea texto.

6. LÓGICA DE NEGOCIO

Crear un servicio (worker):

- Pantalla: `pages/worker/WorkerServicesPricingPage.tsx`.
- Endpoints: `POST/GET/PUT /api/my/services`.
- Campos: `name`, `description`, `basePrice` (USD), `isActive`, `sortOrder`.
- Categoría: heredada del perfil del worker (no por servicio individual).
- Geolocalización: a nivel de profesional (`lat`, `lng`), usada para distancia en búsqueda.
- Tabla: `worker_services` (FK a `workers`).

Contratar (cliente):

- Wizard en `pages/client/BookingPage.tsx` (4 pasos: detalles → dirección → agenda → confirmación).
- Endpoint: `POST /api/bookings`.
- Tipos: `service` (contratación directa) o `inquiry` (consulta).
- Modos de precio: `service` (precio fijo del worker) o `bid` (cliente propone presupuesto).

Estados de booking:

```

pending → accepted → payment_pending → payment_confirmed
      → in_progress → finished → completed
                        ↓
                    disputed → resolved (admin)
                        ↓
                    cancelled

```

Completar:

- Worker marca `finished`. Cliente confirma → dispara rating.
- Cron `autoConfirmFinishedBookings` cierra automáticamente tras 25 minutos sin disputa.
- Reviews bidireccionales: `POST /api/reviews` (cliente→worker), `POST /api/client-ratings` (worker→cliente con tags tipo "Buen pagador").

Generación de ingresos:

- Comisión típica 10% sobre cada transacción.
- `worker_earnings` = monto - comisión, acreditado al pasar a `completed`.
- `store_earnings_amt` análogo para tiendas.
- Plan premium (`client_plan`) y boosts en ranking son fuentes adicionales.

Disputas:

- Booking pasa a `disputed`. Se abre canal de chat privado cliente↔worker↔admin.
- Admin puede mover a `dispute_in_review` y resolver eligiendo `winner: client | worker`.
- Si gana worker → libera pago. Si gana cliente → bloquea/reembolsa.
- Archivos: `routes/disputes.ts`, `dispute_messages` table.

7. MARKETPLACE / TIENDAS

Crear tienda:

- `POST /api/stores`. Campos: branding (logo, banner, color acento), ubicación, datos de pago.
- Owner: usuario con rol `cohost`.
- Dashboard: `pages/cohost/StoreDashboardPage.tsx` (revenue, orders, productos).

Productos:

- `POST /api/products`, atados a `storeId`.
- Tipos: `sale` (venta directa) o `rental` (alquiler vía ServiRent).
- Free tier: límite de 5 productos.
- Tabla: `products` con `cohost_id` (legacy) y `store_id` (actual) coexistiendo.

Instant Store importer:

- Archivos: `services/importer.ts`, `routes/imports.ts`.
- Formatos: CSV y JSON.
- Heurísticas: auto-mapea columnas (`sku` → `externalId`, `precio` → `priceUsd`, etc.).
- Fuentes: texto pegado o URL remota.
- **Seguridad:** SSRF protection — bloquea rangos de IP privadas en imports por URL. (Bien hecho.)
- Tablas: `store_imports`, `import_runs`.
- ⚠ `store_imports.apiKey` se guarda en texto plano (hay TODO `// encrypt at rest` en el código).

Stock y precios:

- `products.stock` (entero), decrementa en órdenes.
- Moneda primaria USD; al cobrar se guarda `bcvRateUsed` (tasa BCV vigente) en booking/order para histórico.

- Updates: `PUT /api/products/:id`.
-

8. BASE DE DATOS

Postgres 16 + Drizzle. Todas las tablas en `lib/db/src/schema/`.

Tablas principales:

Tabla	PK	FKs clave	Rol
users	serial	—	identidad central, multirole
workers	serial	user_id , category_id , cohost_id	perfil profesional
stores	serial	cohost_id	tiendas marketplace
products	serial	store_id , cohost_id	inventario
bookings	serial	client_id , worker_id , service_id	transacciones de servicios
product_orders	serial	product_id , client_id	órdenes de productos
custom_orders	serial	varias	pedidos personalizados
rentals	serial	product_id , client_id , owner_id	alquileres
delivery_requests	serial	product_id , store_id , client_id	logística
bdv_c2p_transactions	serial	polimórfica	pagos C2P
bdv_payment_notifications	serial	credited_user_id	webhooks banco
urgent_requests	serial	client_id	leads urgentes
reviews / client_ratings / product_ratings	serial	varias	reputación bidireccional
chat_messages / store_messages / dispute_messages	serial	varias	mensajería
withdrawals	serial	worker_id	retiros
business_managers / manager_invitations	serial	store_id , user_id	nuevo (Fase 1 gestores)
system_alerts	serial	—	alertas internas (legacy worker sunset)
action_logs / events	serial	polimórficos	auditoría

Relaciones:

- User → Worker (1:0 o 1:1).
- User(cohost) → Store → Product .
- Booking → Review / ClientRating (1:1 por lado).

- `client_favorites` join table cliente↔worker con UNIQUE.
- Polimorfismo blando en `bdv_c2p_transactions(reference_type, reference_id)` y `action_logs(target_type, target_id)`.

Riesgos reales del diseño:

#	Riesgo	Detalle
1	Deuda legacy en <code>users</code>	<code>role</code> + <code>secondaryRole</code> + <code>roles[]</code> coexistiendo
2	Deuda legacy en <code>products</code>	<code>cohost_id</code> (legacy) + <code>store_id</code> (actual)
3	Índices faltantes en FKs hot	<code>workers.category_id</code> , <code>bookings.service_id</code> , <code>bdv_payment_notifications.credited_user_id</code> no tienen índice → escaneos secuenciales
4	JSON guardado como text	<code>stores.payment_details</code> , <code>stores.builder_config</code> , <code>job_profiles.skills</code> — debería ser <code>jsonb</code>
5	Agregados denormalizados sin trigger	<code>workers.rating</code> y <code>workers.review_count</code> son cache de <code>reviews</code> . Si el código se olvida de actualizar, queda inconsistente
6	Snapshots como text en <code>rentals</code>	<code>product_name</code> , <code>owner_name</code> , <code>client_name</code> guardados al momento. OK si es histórico, mal si se espera datos vivos
7	Sin soft-delete global	Mezcla de <code>is_active: boolean</code> y deletes duros. Al borrar un user con bookings, FKs frágiles
8	<code>store_imports.apiKey</code> en texto plano	Marcado TODO en el código
9	<code>serial (int4)</code> en todas las PKs	A muchos millones de filas hay que migrar a <code>bigserial</code> / <code>uuid</code> . Hoy no urge
10	Naming inconsistente de archivos	<code>push-subscriptions.ts</code> con guión; el resto con underscore

9. SEO

Cómo está implementado:

- App es CSR (Client-Side Rendered) — Google ejecuta JS pero hay penalty vs SSR.
- Hook propio `useSeo` (`src/lib/seo-helpers.ts`) que manipula `document.head` directamente (no usan React Helmet).
- Inyecta: `<title>`, `<meta name="description">`, OpenGraph, Twitter Card, `<link rel="canonical">`, `<script type="application/ld+json">`.

Páginas con SEO dinámico real:

- `BlogArticlePage.tsx` — toma `article.metaTitle` y `article.metaDescription` desde DB.
- `PublicWorkerPage.tsx` — genera "Contratar a [Nombre] | [Categoría] en LinkServi".
- `CategoryCityPage.tsx` — combina "[Categoría] en [Ciudad]" para long-tail.

Sitemap (bien hecho):

- `/sitemap.xml` servido dinámicamente por `routes/seo.ts`.
- Incluye páginas estáticas + artículos de blog + pares categoría-ciudad **filtrados** (solo donde hay ≥ 1 worker, evita thin content) + perfiles de worker/job.
- Pesos de prioridad por tipo de URL.

robots.txt:

- En `artifacts/servilink/public/robots.txt`.
- Permite todos los bots, bloquea `/admin`, `/api`, `/worker`.
- Permite explícitamente GPTBot y ClaudeBot (decisión consciente para indexación en LLMs).

Lo que está MAL en SEO:

- **Sin SSR ni prerender** → contenido dinámico (perfiles, categorías) tarda en indexar y compite peor que Mercado Libre o cualquier competidor con SSR.
- **Sin redirects 301 reales en servidor.** Si renombras una categoría, las URLs viejas se pierden.
- **Sin breadcrumbs estructurados (Schema.org BreadcrumbList)** visibles a Google.
- No vi `next/sharp` -style image optimization para Open Graph dinámico (imágenes OG son las que sube el usuario sin redimensionar).
- `useSeo` manipula DOM en `useEffect` → hay un flash inicial donde el `<title>` es el genérico antes de pintar el específico (Google reciente lo maneja bien, pero crawlers viejos no).

Lo que está BIEN:

- Sitemap dinámico bien filtrado.
- JSON-LD soportado por hook.
- Canonical y OG bien manejados por página.
- Robots.txt explícito.

10. PERFORMANCE

Frontend (bien optimizado para SPA):

- **Code splitting agresivo:** todas las rutas con `React.lazy` + `Suspense` en `App.tsx`. Bundles separados por área (admin, client, worker, cohort).
- **Manual chunks** configurados en `vite.config.ts`.
- **TanStack Query** con cache 1min stale + 10min gc → reduce requests duplicados.
- **Service Worker** (`src/sw.ts`, Workbox):
 - `NetworkFirst` (8s timeout) para `/api/`.

- `CacheFirst` para CDN externos (fonts, Leaflet).
- `StaleWhileRevalidate` para imágenes.
- Imágenes con `loading="lazy"`; utilidad `imageUtils.ts` para WebP/AVIF.

Backend (mezcla):

- **Bien:** la mayoría de listados usan `innerJoin` / `leftJoin` para evitar N+1.
- **Bien:** WebSocket con heartbeat y manejo de errores robusto.
- **Mal:** `/api/products` lista todos los activos y ordena en memoria del proceso. Cuello de botella claro.
- **Mal:** `computeSmartScore` en `routes/servicios/workers.ts` corre varias subqueries paralelas por cada profesional listado. A escala incomoda.
- **Mal:** algunos endpoints sin paginación.
- **Bien:** `/api/blog/articles` usa limit/offset.

Cuellos de botella reales (en orden de gravedad):

1. Listado de productos sin pagination + sort en JS.
2. Ranking de workers sin índices ni materialización.
3. Sin Redis adapter para Socket.io → no escala horizontalmente.
4. Sin colas (`BullMQ`) → emails y push corren en el mismo proceso bloqueando event loop.
5. Imágenes sin redimensionado (cliente recibe full-size siempre).

11. SEGURIDAD

Bien:

- `helmet` activo (con ajustes para Replit).
- `bcryptjs` 10 rounds para passwords.
- JWT con expiración 7d, cookie `HttpOnly` + `SameSite=Lax` + Bearer fallback.
- Password reset con tokens de alta entropía + SHA-256 en DB + 30min de expiración.
- Rate limiting: `authRateLimiter` (30/15min en login/register), `createRateLimiter` (20/min en bookings).
- CORS restringido a `.replit.dev` / `.replit.app` /localhost.
- `SESSION_SECRET` validado al startup.
- Subidas de archivos vía signed URLs GCS (no buffer en memoria).
- Drizzle = queries parametrizadas → **bajo riesgo de SQL injection** (incluso en los pocos `sql\` ``` que usan template literals correctamente).
- SSRF protection en importer.

Mal / Riesgos confirmados:

#	Severidad	Riesgo	Archivo / Línea
1	Alta	IDOR en <code>GET /api/bookings/:bookingId</code> — sin ownership check	<code>routes/servicios/bookings.ts:341</code>
2	Alta	<code>store_imports.apiKey</code> en texto plano en DB	<code>lib/db/src/schema/imports</code>
3	Media	XSS no sanitizado server-side en contenido de usuario (descripciones, nombres). Depende 100% del front	varios
4	Media	Sin CSRF token (mitigado por SameSite + Bearer, pero si el front tiene XSS, los cookies/Bearer son robables)	<code>lib/auth.ts</code>
5	Media	Validación con Zod inconsistente: muchas rutas hacen <code>`if (!name</code>	
6	Media	<code>POST /api/products/:id/track-view</code> público sin rate limit por producto/IP — inflexible	<code>routes/marketplace/products.ts</code>
7	Baja	Sin refresh tokens — invalidación de sesión solo por expiración	<code>lib/auth.ts:31</code>
8	Baja	Logs <code>pino</code> no enmascaran emails ni teléfonos en cuerpos de request	<code>app.ts</code>

12. PROBLEMAS REALES (CONSOLIDADO)

Bugs / código frágil:

- IDOR en `bookings/:bookingId` (item #1 de seguridad).
- Tres columnas de rol coexistiendo (`role` , `secondaryRole` , `roles[]`) — cualquier código nuevo que olvide leer las tres genera bugs invisibles.
- `products` con `cohost_id` legacy + `store_id` nuevo, ambos nullables.
- Agregados (`workers.rating` , `review_count`) sin trigger DB → desincronizables.

Código muerto / deuda:

- `pages/client/UrgentRequestPage` con feature deprecada (acabaste de remover el card pero la página sigue).
- `users.role` y `users.secondaryRole` siguen usados por código viejo.
- Página de modo urgencia ya no aparece en home, pero sí en sidebar.
- `naming` mezclado en filenames (`push-subscriptions.ts` vs `bdv_payments.ts`).

Riesgos operativos:

- Sin colas → si Resend cae, los emails fallan en línea con el request HTTP.
- Sin Redis → Socket.io es single-node.
- `applyDomainEffect` dentro del request al banco → si el usuario cierra el browser, dependes del webhook (mitigado por conciliación pero no perfecto).
- Sin observability más allá de pino logs (sin Sentry, sin métricas Prometheus, sin tracing).
- Sin tests automatizados (no encontré directorio `tests/` con Vitest/Jest configurado para ejecutar en CI).

Malas prácticas:

- Validación de inputs inconsistente (Zod usado para schema pero muchos endpoints validan manual).
- JSON guardado como `text` en columnas (`payment_details` , `builder_config`).
- Falta índices en FKs caliente (`category_id` , `service_id`).
- Logs sin redacción de PII.
- `apiKey` de imports en texto plano.

13. FORTALEZAS

Está bien hecho:

- **Sistema de pagos C2P:** advisory locks, recálculo server-side del monto, webhooks con UNIQUE, anulación automática en caso de fallo post-débito. Es de las piezas más sólidas del repo.
- **Escrow + auto-confirm 25 min:** patrón profesional, evita disputas eternas.
- **Sitemap dinámico filtrado** para evitar thin content.
- **Code splitting agresivo + Service Worker multi-estrategia.**
- **Reputación bidireccional** (cliente califica worker y viceversa con tags) — diferenciador frente a competencia.
- **OpenAPI + Orval + Zod compartido** — el contrato API está versionado y autogenerado, reduce drift entre back y front.
- **Disputas con chat privado y resolución por admin** — flujo completo, no parchado.
- **Multi-rol nativo** (`roles[]`) + RBAC con permisos JSON para gestores — diseño preparado para Super-App.
- **Importer de tiendas con CSV/JSON/URL + SSRF protection** — feature de adquisición fuerte.

Escalable:

- Monorepo con shared schema = cambio de DB se propaga a back y front con tipos.
- TanStack Query + ServiceWorker = capa de cache decente sin Redis.
- Endpoints REST claros y separados por dominio (`servicios/` , `marketplace/` , `empleo/` , `alquileres/`).

Diferencial:

- C2P automatizado con BDV es tecnología que pocas startups venezolanas implementan bien.
- Multi-vertical en una sola app (servicios + tienda + alquileres + empleo + delivery).
- AI bots (GPTBot, ClaudeBot) explícitamente permitidos → posicionamiento en respuestas LLM.

14. RECOMENDACIONES

Prioridad MÁXIMA (riesgo de seguridad o pérdida de datos):

1. Arreglar IDOR en `GET /api/bookings/:bookingId` — agregar check de ownership o cohost o admin.
2. Cifrar `store_imports.apiKey` en reposo (KMS o `pgcrypto`).
3. Resolver la deuda de `users.role` / `secondaryRole` / `roles[]`: migrar todo a `roles[]` y borrar las columnas viejas en una sola migración + deprecation flag.
4. Agregar índices a `workers.category_id`, `bookings.service_id`, `bdv_payment_notifications.credited_user_id`.

Prioridad ALTA (técnica, escalabilidad inminente): 5. Mover sort de `/api/products` a SQL con paginación obligatoria. 6. Convertir columnas JSON-en-text a `jsonb` (`stores.payment_details`, `stores.builder_config`, `job_profiles.skills`). 7. Agregar Sentry o equivalente para captura de errores en producción. 8. Sanitizar contenido user-generated server-side (DOMPurify o equivalente para descripciones). 9. Implementar refresh tokens y logout server-side (revocación de JWT).

Prioridad MEDIA (preparar para escala 10x): 10. Introducir Redis para: cache de ranking + Socket.io adapter + rate limit distribuido. 11. Introducir BullMQ (o similar) para emails, push, conciliación BDV → desacopla de event loop. 12. Materializar ranking de workers (vista materializada o tabla cache) refrescada cada N minutos. 13. CDN delante (Cloudflare): cache de assets + WAF + DDoS. 14. Image transformation pipeline (Cloudflare Images, Imgix, o GCS + serverless function).

Prioridad MEDIA (producto): 15. SSR o prerender para páginas SEO críticas (blog, profesional público, categoría-ciudad). Next.js no es necesario; basta con `vite-ssg` o un worker que pre-renderice. 16. Breadcrumbs Schema.org en perfiles y categorías → rich results en Google. 17. Open Graph dinámico con imágenes generadas (no la del usuario crudo). 18. Tests automatizados E2E con Playwright en CI (cubrir: registro, booking, pago C2P mock, retiro).

Prioridad BAJA (limpieza): 19. Borrar página `UrgentRequestPage` y entradas de sidebar relacionadas si la feature ya no se ofrece. 20. Renombrar `push-subscriptions.ts` a `push_subscriptions.ts` para consistencia. 21. Documentar el state machine de bookings en un README dentro de `routes/servicios/`. 22. Trigger DB (o lógica centralizada) que mantenga `workers.rating` y `review_count` en sincronía.

Conclusión técnica honesta:

LinkServi es una codebase **madura para una startup en early stage**, con piezas serias (pagos, escrow, OpenAPI, multi-rol, sitemap dinámico) y deuda real pero acotada (IDOR puntual, columnas legacy, falta de colas y CDN). El sistema **soporta hoy** la operación a la escala que tienes y, con las recomendaciones de prioridad alta resueltas, soporta sin reescritura un crecimiento de **10x** en usuarios. Una reescritura no es necesaria; lo que se necesita es **disciplina de mantenimiento** y resolver la deuda en orden.